Канонизација међукода (свођење на основни облик)

- Неки аспекти међукода не одговарају машинском језику, а неки други аспекти ометају оптимизацију међукода.
- Посебан проблем су ESEQ и SEQ чворови који садрже исказе доделе и обављају У/И операције (то се назива *ивичним ефектима*).
- *Ивични ефекти* доводе до тога да се подизрази неког израза не могу израчунавати у било ком редоследу.

Неслагања међукода и машинског кода

- 1. ESEQ и SEQ изазивају да различити редоследи израчунавања подизраза дају различите резултате целог израза

Неслагања међукода и машинског кода

- 2. CALL у оквиру аргумената других CALL
  - Проблем: смештање аргумената у фиксан број регистара за формалне параметре

Дигресија: позивна конвенција

- Позивна конвенција одређује спрегу између позване и позивајуће функције.
- Дефинише две ствари:
  - Начин преношења стварних параметара и повратне вредности
    - У које ресурсе се смештају
      - Ако у регистре: веза између редног броја параметра и конкретног регистра
      - Ако на стек: веза између редног броја параметра и редоследа на стеку
    - Договор где се који параметар смешта зависи од његовог редног броја и типа
  - Ко које ресурсе сме да дира (мења)
    - Који регистри ће гарантовано имати исту вредност пре и после позива, а који не
    - Ко ће заузимати и ослобађати меморију на стеку за параметре и повратну вредност

Неслагања међукода и машинског кода

- 2. CALL у оквиру аргумената других CALL
  - Проблем: смештање аргумената у фиксан број регистара за формалне параметре

```
int foo(int x, int y);

a = foo(5, 6);

r0 <- 5
r1 <- 6
call _foo
mem[_a] <- r0
```

```
int bar(int x);

a = foo(bar(5), 6);

r0 <- 5
//r1 <- 6
call _bar
r1 <- 6
call _foo
mem[_a] <- r0
```

```
int baz(int x);

a = foo(bar(5), baz(6));

r0 <- 5
call _bar
// r0 <- 6
r2 <- r0
r0 <- 6
call _baz
r1 <- r0
r0 <- r2
call _foo
mem[_a] <- r0
```

По позивној конвенцији први параметар мора бити у r0, а други у r1. Повратна вредност је у r0. Такође, и вредност регистра r1 може бити промењене унутар функције.

Трансформација међукода

- Проблеми се елиминишу трансформацијом међукода, где се стабло претвара у листу тзв. каноничких стабала, која немају ни SEQ ни ESEQ чворова (а немају ни CALL чворове којима је CALL предак).

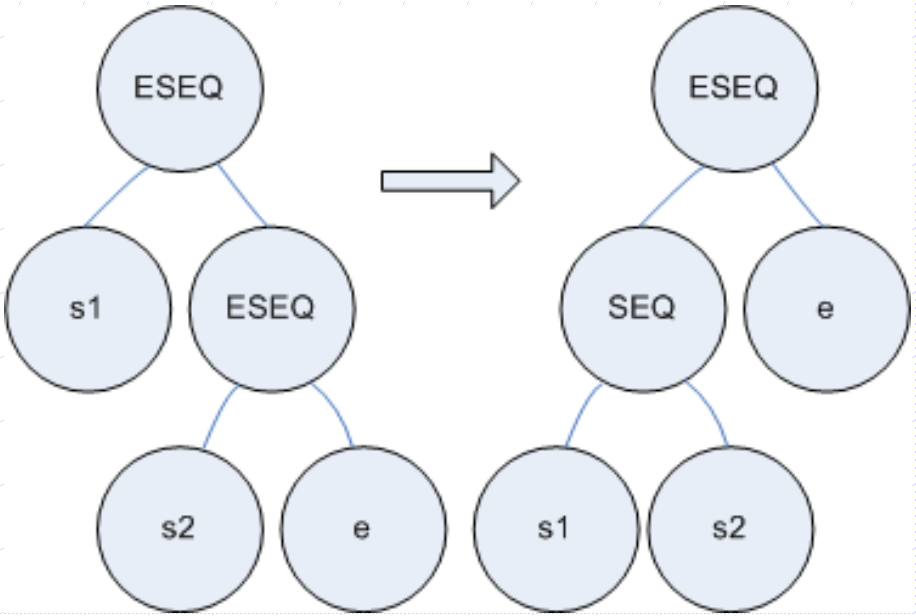
Каноничка стабла међукода

- Две особине:
  - не садржи SEQ (низ од два исказа) и ESEQ (низ од једног исказа и једног израза) чворове, и
  - предак CALL (позив функције) чвора је чвор EXP (израз) или MOVE (пребаци резултат израза у задату локацију)
- Како уклонити ESEQ?
  - Идеја: подизати их навише док не постану SEQ

I група правила претварања

$ESEQ(s_1,ESEQ(s_2,e)) = ESEQ(SEQ(s_1,s_2),e)$

- Директно из дефиниције ESEQ(s, e):
  - „изврши прво исказ s, због ивичних ефеката, а затим израчунај вредност израза e”



II група правила претварања

BINOP(op, ESEQ(s, e_1), e_2) =
ESEQ(s, BINOP(op, e_1, e_2))

MEM(ESEQ(s, e)) = SEQ(s, MEM(e))

JUMP(ESEQ(s, e)) = SEQ(s, JUMP(e))

CJUMP(op, ESEQ(s, e_1), e_2 , l_t , l_f) =
SEQ(s, CJUMP(op, e1, e2, l_t, l_f))

- Директно следе из дефиниција BINOP, MEM, JUMP i CJUMP

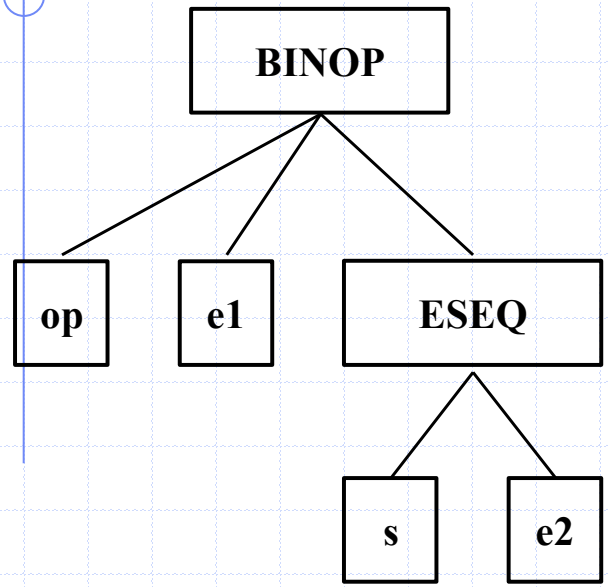
III група правила претварања

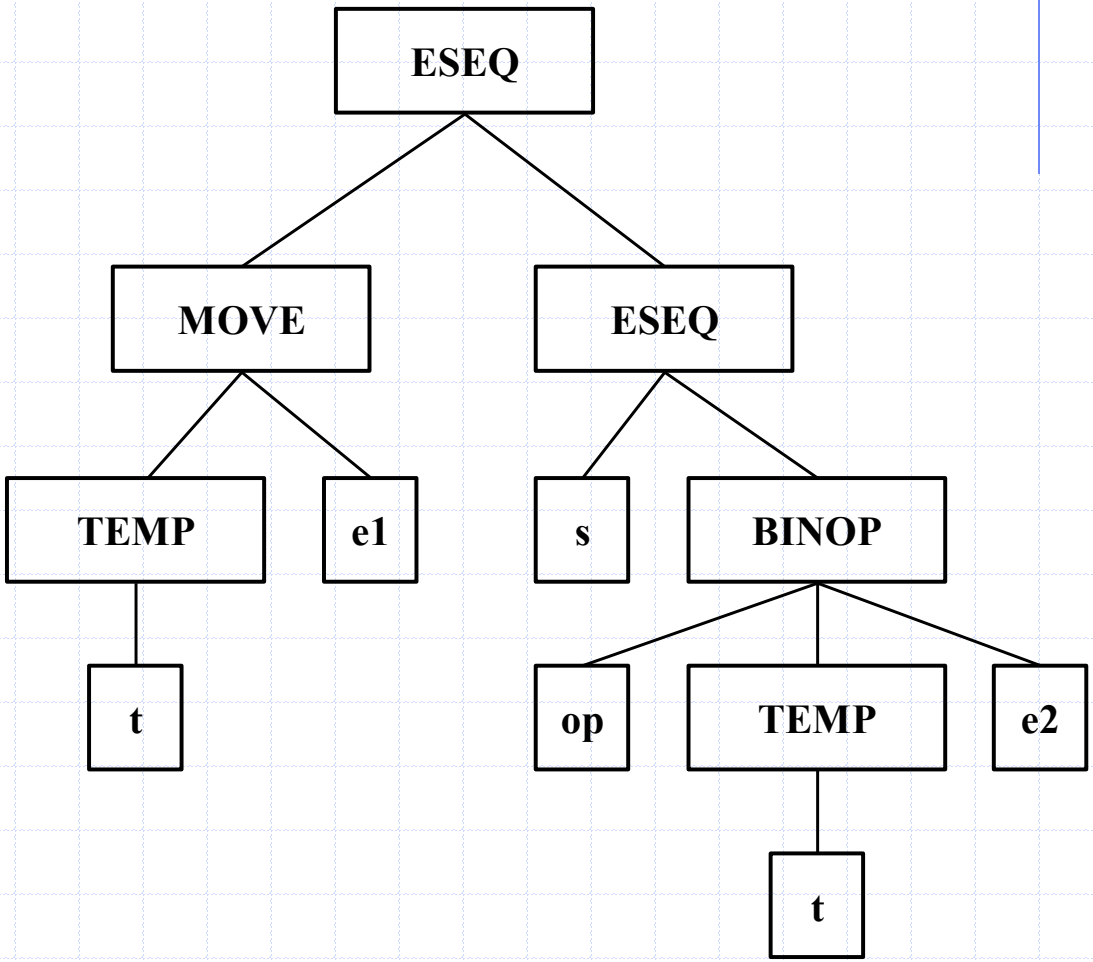
BINOP(op, e_1, ESEQ(s, e_2)) =
ESEQ(MOVE(TEMP t, e_1), ESEQ(s, BINOP(op, TEMP t, e_2)))

CJUMP(op, e_1, ESEQ(s, e_2), l_t, l_f) =
SEQ(MOVE(TEMP t, e_1) , SEQ(s, CJUMP(op, TEMP t, e_2, l_t, l_f)))

- Да би се спречили ивични ефекти s на e1,
  - вредност e_1 се смешта у привремену променљиву t

Илустрација I правила из III групе





IV група правила претварања

- У случају да s нема ивичних ефеката на e1 горња правила се могу упростити:

$$BINOP(op, e_1, ESEQ(s, e_2)) = ESEQ(s, BINOP(op, e_1, e_2))$$

$$CJUMP(op, e_1, ESEQ(s, e_2), l_t, l_f) = SEQ(s, CJUMP(op, e_1, e_2, l_t, l_f))$$

Извлачење CALL чворова

- Извлачење CALL на почетак подизраза
  - непосредна замена CALL чвора инструкцијом за позив потпрограма није могућа
    - дошло би до преписивања процесорског регистра за смештање повратне вредности
    - Нпр. BINOP(PLUS, CALL(...), CALL(...)), први CALL би преписао повратну вредност другог
- Решење: свака повратна вредност се додељује новом привременом регистру
  - CALL(fun, args) => ESEQ(MOVE(TEMP t, CALL(fun, args)), TEMP t)

Вишак MOVE чворова

- Надаље ће ESEQ извлачити MOVE ван BINOP и других израза
  - Овај поступак доводи до вишка MOVE операција
    - који се елиминишу касније у фази доделе регистара

Линеаризација међукода

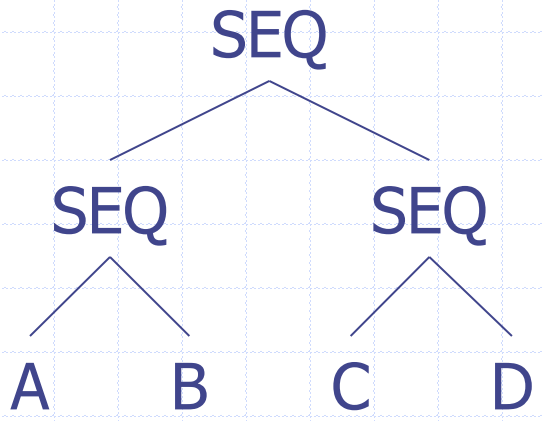
- На крају се долази до стабла у ком су сви SEQ на врху стабла
  - Линеаризација се обавља применом правила:

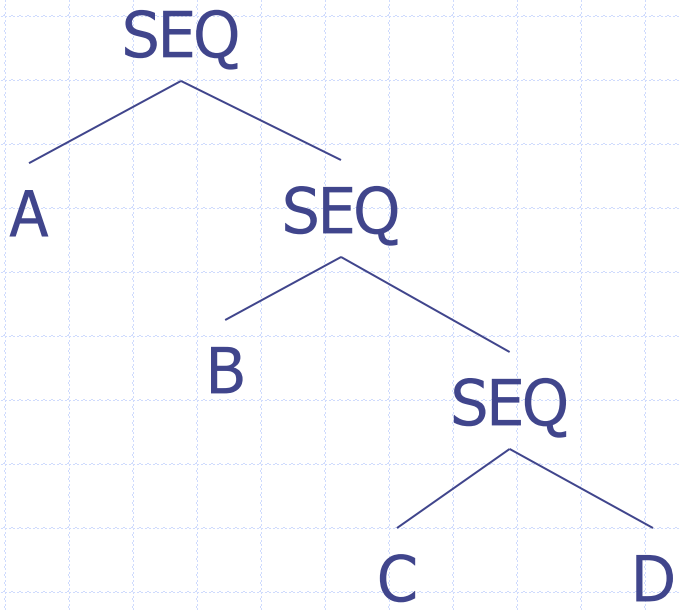
    SEQ(SEQ(a, b), c) = SEQ(a, SEQ(b, c))
  - Тако се долази до линеаризованог израза:

    $SEQ(s_1, SEQ(s_2, \ldots , SEQ(s_{n-1}, s_n) \ldots))$
  - SEQ више не даје никакву структурну информацију и може се уклонити – добија се проста листа:

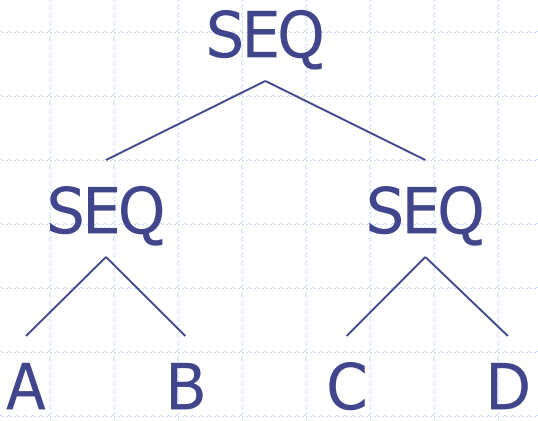
    $s_1, s_2, \ldots , s_{n-1}, s_n$

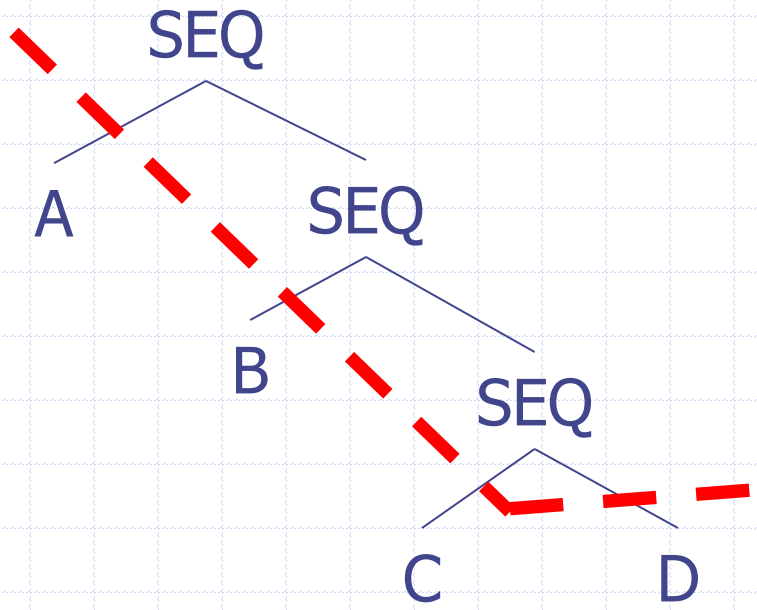
Линеаризација међукода





Линеаризација међукода





A, B, C, D

Основни блок (енгл. basic block)

- Основни програмски блок се дефинише као низ суседних исказа који:
  - Не садрже лабелу, осим можда на самом почетку, и
  - Не садрже скокове, осим можда на самом крају
- Другим речима, основни блок је низ суседних исказа који се увек извршавају у пакету, без скокова и могућих ускока.
- Свака лабела представља почетак основног блока, а сваки скок представља његов крај. Али обрнуто не важи: нпр. на почетку блока не мора бити лабела, довољно је да је претходни исказ скок.

Анализа тока управљања

- У циљу одређивања основних блокова програма анализира се ток управљања
  - Инструкција условног гранања
    - се посматра тако да се са ње може скочити било на лабелу t или f
      - све инструкције које нису инструкције безусловног или условног скока су безначајне
  - Низови инструкција између инструкција гранања се групишу у блокове
    - Даље се анализира искључиво ток управљања између основних блокова

Одређивање основних блокова и трагова (енгл. trace)

- IR стабло се најпре подели на основне блокове.
- Они се затим распоређују по програмским траговима (енгл. *trace scheduling*), тако да се добије код са што мање инструкција скока.
- Сваки основни блок је покривен искључиво једним трагом.

Распоређивање блокова (1/2)

- Након што су основни блокови одређени,
  - могу се распоредити у било ком редоследу,
    - без утицаја на резултат извршења програма
- Неки распореди су бољи од других
  - Зависно од инструкција циљног процесора и додатно расположивих информација

Распоређивање блокова (2/2)

- Увек се тражи распоред који максимизира број безусловних скокова праћених лабелом на коју се скаче,
  - пошто се ти безусловни скокови као сувишни могу избацити из програма

Трагови (traces)

- *Трагом* се назива низ исказа који се **могу** узастопно извршавати током извршавања програма.
- Траг може садржати и условне скокове.
- Од интереса је **минималан** скуп трагова који се међусобно не преклапају, а у потпуности прекривају програм.

Алгоритам за одређивање мин. скупа трагова

Сместити све основне блокове програма у листу Q;

Понављај док Q није празна

Започни нов (празан) траг T;

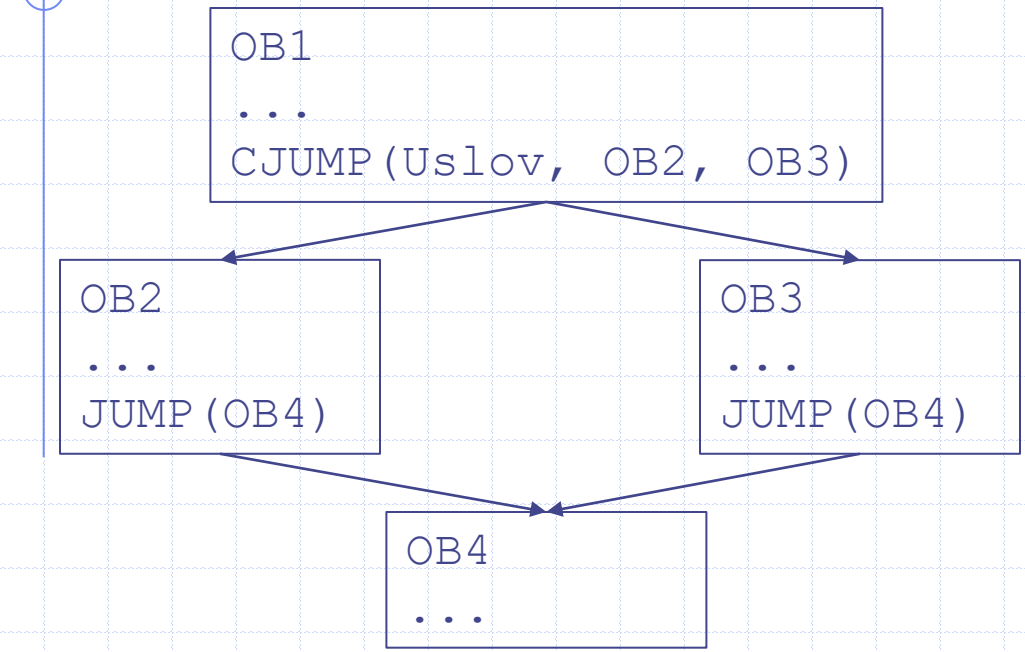
Изланчај елемент B са почетка листе Q;

Понављај док B није обележен

Обележи B; Додај B на крај трага T; Испитај све наследнике B (блокови на које он скаче);

Ако постоји необележен наследник C онда изврши доделу B = C; (Кључна ствар: ако има више наследника – који изабрати?)

Крај текућег трага T;

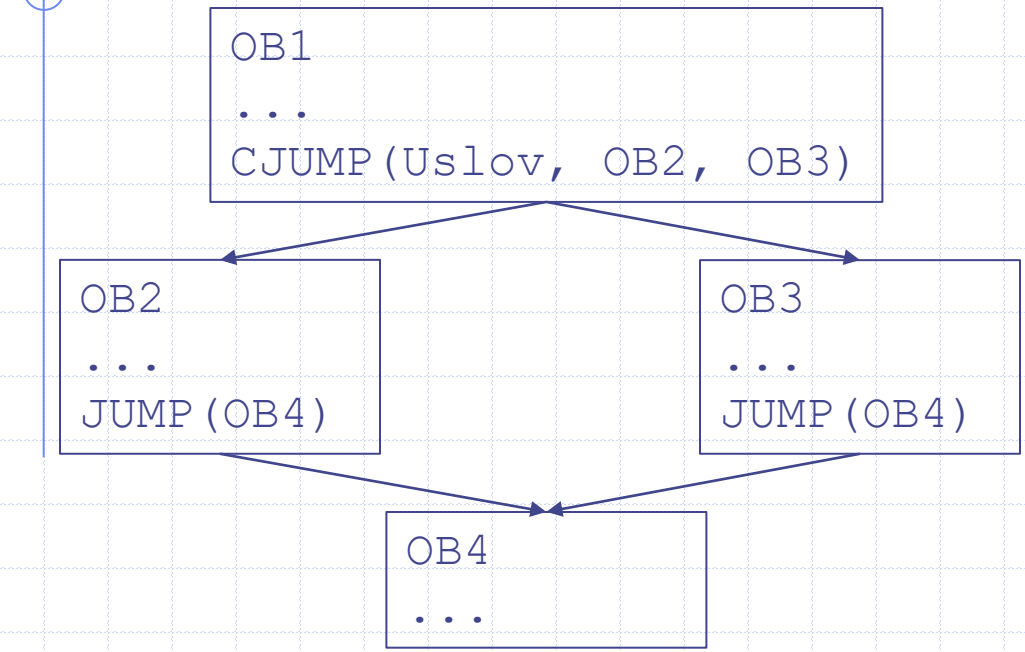



```
OB1
...
CJUMP(Uslov, OB2, OB3)
```

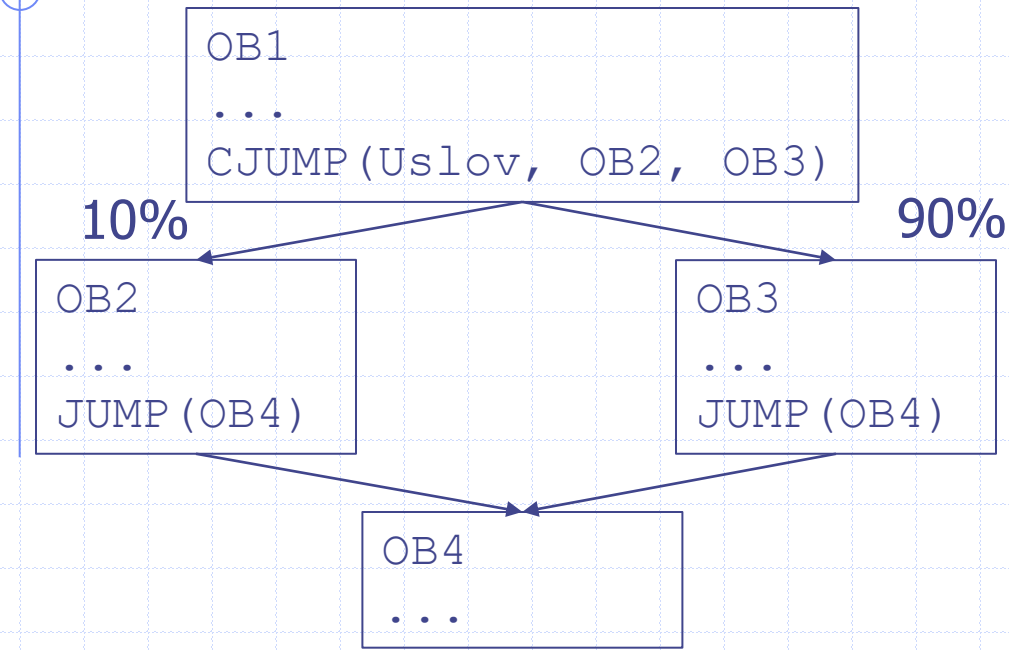
```
OB4
...
```

```
OB2
...
JUMP(OB4)
```

```
OB3
...
JUMP(OB4)
```




```
OB1
...
CJUMP(Uslov, OB2, OB3)
OB2
...
JUMP(OB4)
OB4
...
OB3
...
JUMP(OB4)
```




```
OB1
...
CJUMP(Uslov, OB2, OB3)
```

```
OB3
...
JUMP(OB4)
```

```
OB4
...
```

```
OB2
...
JUMP(OB4)
```

Оптимални трагови (1/2)

- Често извршаван низ инструкција, свој траг
  - нпр. тело петље
  - минимизира се број безусловних скокова, а омогућавају се и оптимизационе технике:
    - доделе регистара и
    - распоређивања инструкција
  - Следи пример

Оптимални трагови (2/2)

- Пример два начина организовања програма по траговима.
  - Десна варијанта је боља јер нема JUMP чвора у телу while петље

<table>
<tr><th>Прва варијанта</th><th>Друга варијанта</th></tr>
<tr><td rowspan="2">искази пролога
JUMP(NAME test)
LABEL(test)
CJUMP(>, i, N, kraj, telo)
LABEL(telo)
искази тела петље
JUMP(NAME test)</td><td>искази пролога
JUMP(NAME test)</td></tr>
<tr><td>LABEL(telo)
искази тела петље
<s>JUMP(NAME test)</s> ← бриши
LABEL(test)
CJUMP(>, i, N, kraj, telo)</td></tr>
<tr><td>LABEL(kraj)
искази епилога</td><td>LABEL(kraj)
искази епилога</td></tr>
</table>